

Ausgangssituation

Eine Mediathek verwaltet unterschiedliche Medien. Es gibt zwei Arten von Medien:

- `Buch`
- `Film`

Die Medien werden aus einer CSV-Datei gelesen, verarbeitet, mit Lambda-Ausdrücken gefiltert und wieder in CSV-Dateien geschrieben.

Diese Übung konzentriert sich auf drei Themen:

- eigene Exceptions und Exception Handling
- File Handling mit CSV-Dateien
- Lambdas mit einem funktionalen Interface

Vorgegebenes Projekt

Im Ordner `Vorgabe/src/main/java` befindet sich ein Codegerüst. Implementieren Sie die Stellen, die im Code mit `TODO` markiert sind.

Zwei Dateien sind absichtlich noch nicht vorhanden und müssen von Ihnen selbst erstellt werden:

- `MedienException.java`
- `MedienFilter.java`

Einige einfache Listenmethoden sind bereits fertig implementiert. Sie dienen nur zum Testen und gehören nicht zum Schwerpunkt dieser Übung.

Die Datei `Vorgabe/src/main/resources/medien.csv` ist als Beispieldatei vorbereitet. Sie enthält gültige und ungültige Zeilen. Ungültige Zeilen sollen beim Einlesen nicht zum Programmabbruch führen.

Allgemeine Anforderungen

- Arbeiten Sie mit normalen Schleifen.
- Fehler in Model-Klassen werden mit `MedienException` gemeldet.
- Fehlerhafte CSV-Zeilen dürfen das Programm nicht abbrechen.
- Lambda-Ausdrücke sollen über das funktionale Interface `MedienFilter` verwendet werden.

Teil 1: Exceptions in den Model-Klassen

Erstellen Sie zuerst die eigene Exception-Klasse `MedienException`. Implementieren Sie danach die Validierung in den `TODO`-Methoden der Klassen `Medium`, `Buch` und `Film`.

Titel prüfen

```
public void setTitel(String titel) throws MedienException
```

Anforderungen:

- `titel` darf nicht `null` sein.
- `titel` darf nicht leer sein.
- Der gespeicherte Titel soll keine führenden oder abschließenden Leerzeichen enthalten.
- Bei ungültigem Wert wird eine `MedienException` mit sinnvoller Fehlermeldung geworfen.

Erscheinungsjahr prüfen

```
public void setErscheinungsjahr(Year erscheinungsjahr) throws MedienException
```

Anforderungen:

- `erscheinungsjahr` darf nicht `null` sein.
- `erscheinungsjahr` muss später als 2000 liegen.
- Bei ungültigem Wert wird eine `MedienException` geworfen.

Bewertungen prüfen

```
public void setBewertungen(int bewertungen) throws MedienException
```

Anforderungen:

- `bewertungen` muss größer oder gleich 0 sein.
- Bei ungültigem Wert wird eine `MedienException` geworfen.

Buchdaten prüfen

```
public void setSeiten(int seiten) throws MedienException
```

Anforderungen:

- `seiten` muss größer als 0 sein.
- Bei ungültigem Wert wird eine `MedienException` geworfen.

Filmdaten prüfen

```
public void setDauerMinuten(int dauerMinuten) throws MedienException
```

Anforderungen:

- `dauerMinuten` muss größer als 0 sein.
- Bei ungültigem Wert wird eine `MedienException` geworfen.

Beliebtheit berechnen

```
public double berechneBeliebtheit()
```

Berechnen Sie die Beliebtheit in den Unterklassen:

- `Buch: bewertungen * 1.5`
- `Film: bewertungen * 2.0`

Die Beliebtheit wird später in Lambda-Filtern verwendet.

Medium aufnehmen

```
public boolean aufnehmen(Medium medium) throws MedienException
```

Anforderungen:

- `medium` darf nicht `null` sein.
- Das gleiche Medium darf nicht bereits in der Liste enthalten sein.
- Verwenden Sie zum Prüfen auf Duplikate die vorhandene `equals`-Methode.
- Bei `null` oder Duplikat wird eine `MedienException` geworfen.
- Bei erfolgreichem Einfügen wird `true` zurückgegeben.

Teil 2: CSV-Umwandlung

Diese Methoden werden für den Dateiimport und Dateiexport benötigt.

Gemeinsame CSV-Felder erzeugen

```
public String toCsvString()
```

In `Medium` liefert diese Methode die gemeinsamen CSV-Felder ohne Medientyp:

```
Titel;Erscheinungsjahr;Bewertungen
```

Beispiel:

```
Java Schritt für Schritt;2025;18
```

Vollständige CSV-Zeilen erzeugen

```
public String toCsvString()
```

In **Buch**:

```
Buch;Titel;Erscheinungsjahr;Bewertungen;Seiten
```

In **Film**:

```
Film;Titel;Erscheinungsjahr;Bewertungen;DauerMinuten
```

Teil 3: File Handling und Exception Handling

Implementieren Sie die TODO-Methoden für CSV-Import und CSV-Export in **Medienverwaltung**.

Die Dateipfade werden nicht als Methodenparameter übergeben. Legen Sie dafür in **Medienverwaltung** folgende **private static final** Attribute an:

```
private static final String INPUT_FILEPATH = "src/main/resources/medien.csv";
private static final String OUTPUT_FILEPATH =
"src/main/resources/medien_output.csv";
private static final String REVERSED_OUTPUT_FILEPATH =
"src/main/resources/medien_output_reversed.csv";
private static final String POPULAR_OUTPUT_FILEPATH =
"src/main/resources/medien_output_popular.csv";
private static final String FILME_OUTPUT_FILEPATH =
"src/main/resources/medien_output_filme.csv";
```

CSV-Datei einlesen

```
public void readFromCsv()
```

Anforderungen:

- Leere Zeilen sollen übersprungen werden.
- Jede nicht leere Zeile wird mit `parseMedium(line)` in ein Medium umgewandelt.

- Gültige Medien werden mit `aufnehmen(...)` in die Liste eingefügt.
- Fehlerhafte Zeilen werden übersprungen.
- Das Programm darf wegen einer fehlerhaften CSV-Zeile nicht abbrechen.
- Geben Sie bei fehlerhaften Zeilen eine kurze Fehlermeldung aus.
- Datei-Fehler werden abgefangen und mit einer kurzen Fehlermeldung ausgegeben.

CSV-Zeile parsen

```
private Medium parseMedium(String line) throws MedienException
```

Anforderungen:

- Die übergebene Zeile wird aufgeteilt.
- Die Daten werden in das korrekte Medium umgewandelt.
- Das erzeugte Medium wird zurückgegeben.
- Bei einem Fehler wird eine `MedienException` geworfen.

CSV-Formate:

```
Buch;Titel;Erscheinungsjahr;Bewertungen;Seiten  
Film;Titel;Erscheinungsjahr;Bewertungen;DauerMinuten
```

Alle Medien schreiben

```
public void writeToCsv()
```

Anforderungen:

- Schreiben Sie alle Medien in die Datei `OUTPUT_FILEPATH`.
- Schreiben Sie jedes Medium im CSV-Format in eine eigene Zeile.
- Datei-Fehler werden abgefangen und mit einer kurzen Fehlermeldung ausgegeben.

Medien in umgekehrter Reihenfolge schreiben

```
public void writeReversedToCsv()
```

Anforderungen:

- Schreiben Sie alle Medien in die Datei `REVERSED_OUTPUT_FILEPATH`.
- Beginnen Sie beim letzten Listenelement.
- Schreiben Sie jedes Medium im CSV-Format in eine eigene Zeile.
- Die Reihenfolge der Liste im Speicher darf nicht verändert werden.
- Datei-Fehler werden abgefangen und mit einer kurzen Fehlermeldung ausgegeben.

Teil 4: Lambdas und funktionales Interface

Erstellen Sie zuerst das funktionale Interface `MedienFilter`. Die Datei ist in der Vorgabe noch nicht vorhanden.

Anforderungen:

- Ergänzen Sie die Annotation eines funktionalen Interfaces.
- Erstellen Sie die Methode `boolean akzeptiert(Medium medium);`.
- Die Methode entscheidet, ob ein Medium in ein Filterergebnis aufgenommen wird.

Implementieren Sie danach die TODO-Methoden, die mit diesem Interface arbeiten.

Medien filtern

```
public List<Medium> filtere(MedienFilter filter)
```

Anforderungen:

- Erstellen Sie eine neue `LinkedList<Medium>`.
- Durchlaufen Sie die gespeicherten Medien mit einer normalen Schleife.
- Rufen Sie für jedes Medium `filter.akzeptiert(medium)` auf.
- Wenn der Filter `true` liefert, wird das Medium in die Ergebnisliste aufgenommen.
- Die ursprüngliche Liste darf nicht verändert werden.
- Wenn kein Medium passt, wird eine leere Liste zurückgegeben.

Medien mit Filter zählen

```
public int zaehleMitFilter(MedienFilter filter)
```

Anforderungen:

- Durchlaufen Sie die gespeicherten Medien mit einer normalen Schleife.
- Rufen Sie für jedes Medium `filter.akzeptiert(medium)` auf.
- Zählen Sie alle Medien, für die der Filter `true` liefert.
- Wenn kein Medium passt, wird `0` zurückgegeben.

Gefilterte Medien in Datei schreiben

```
public void writePopularToCsv(MedienFilter filter)
public void writeFilmeToCsv(MedienFilter filter)
```

Anforderungen:

- Schreiben Sie die passenden Medien in die jeweils vorgesehene Datei.
- `writePopularToCsv(...)` schreibt in `POPULAR_OUTPUT_FILEPATH`.
- `writeFilmeToCsv(...)` schreibt in `FILME_OUTPUT_FILEPATH`.
- Schreiben Sie nur Medien, für die `filter.akzeptiert(medium)` den Wert `true` liefert.
- Verwenden Sie für jede Zeile `medium.toCsvString()`.
- Nach jeder Zeile soll ein Zeilenumbruch geschrieben werden.
- Die ursprüngliche Liste darf nicht verändert werden.
- Datei-Fehler werden abgefangen und mit einer kurzen Fehlermeldung ausgegeben.

Lambda-Ausdrücke testen

Erstellen Sie in `Main` mindestens drei Lambda-Ausdrücke und testen Sie damit `filtere(...)`, `zaehleMitFilter(...)`, `writePopularToCsv(...)` und `writeFilmeToCsv(...)`.

Beispiel:

```
MedienFilter beliebt = medium -> medium.berechneBeliebtheit() > 50.0;
```

Teil 5: Datei-Aufgabe

Lesen Sie die Datei:

```
src/main/resources/medien.csv
```

Schreiben Sie danach:

- alle gültig eingelesenen Medien nach `src/main/resources/medien_output.csv`
- alle Medien in umgekehrter Reihenfolge nach `src/main/resources/medien_output_reversed.csv`
- alle Medien mit Beliebtheit größer als `50.0` nach `src/main/resources/medien_output_popular.csv`
- alle Filme nach `src/main/resources/medien_output_filme.csv`

Beim Einlesen sollen ungültige Zeilen erkannt, gemeldet und übersprungen werden.